

**COMP 3015, BCIT Computing
Final Exam Part 1**

Marked out of 50 points

GOOD LUCK!

1. (7 marks) Describe how passwords should be stored. What is the name of a cryptographic function that can provide this functionality, and what do the “cost” and “salt” do / why are they needed?

Passwords should be stored using an encryption algorithm to safely store passwords within a DB. An example of a cryptographic function that could provide options is BCrypt, which takes in a parameter of “cost” to determine how long a password takes to compute (10 is the usual minimum). “Salt” is the term used to describe how a password is encrypted using a unique hash. This way, 2 plain text passwords that are the same, don’t result in the same encrypted passwords.

2. (8 marks) Explain how SQL injection works and why it is a serious security risk in server-side applications. Then, describe how using parameterized prepared statements mitigates this risk. In your answer, include an example of vulnerable code followed by a secure version of the same logic.

SQL injection works by inserting SQL quarriable code within a client’s POST request, with can obfuscate the SQL Server into thinking it’s being asked for a certain query. An example is a query asking for a matching username from the client and then inserting an “OR” statement to process a malicious query to gain access to the entire username of DB data. This is a serioius risk as it pertains to Database Structural Security and dangers. The way to prevent said methods is to provide 2 separate calls to the DB, one as a Prepare Statement from the Server and the other as an Execute Statement. This creates a failsafe to ensure the Prepare Statement is following the allowances of what a client’s POST request should provide, before executing.

```
$username = (GET("username"));  
$sql = "SELECT * username WHERE username = " + username + ';
```

echo \$sql;

Inserting "" OR 1=1" into the username field prompt can give you access to all Usernames in the DB

Preventing this method of attack can be as simple as inserting an if/else check to minimize the number of ways a user can input their request. Also making 2 separate Statements (Prepare/Execute) will ensure a bit more safety.

3. (10 marks) You notice that when you log in to a web application, the session ID held in the cookie set by the application does not change. Additionally, the cookie does not have any flags set.
- a. Discuss what this means for the security of the application.
 - b. What risks are introduced by the lack of cookie flags?
 - c. Provide specific recommendations to improve the application's session management and cookie handling practices.
-
- a) Security of the application could be misused and exploited by many means if the Session ID is held within the cookies without any flags set.
 - b) Risks that could be involved are an external user managing to gain access using the same session ID to get any data the original user had.
 - c) Ways to improve security could be using a method to Set Cookies to expire once the user is done accessing the site, to prevent future risks of exploitation. Another is when the user comes back to the web application, a different Session ID is handed to them to prevent using the same Session IDs.

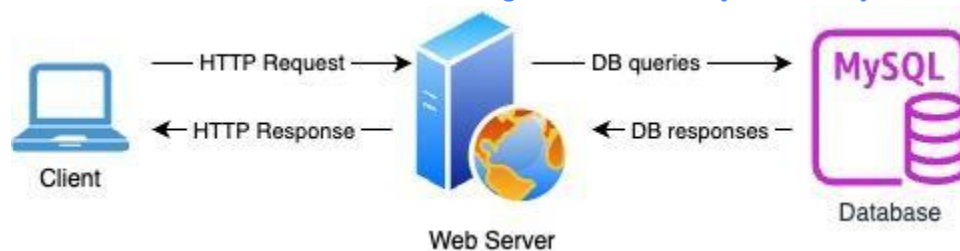
4. (5 marks) Explain the Post/Redirect/Get pattern and why using it is considered a best practice in handling form submissions. Provide a specific example where failing to properly implement this pattern could cause problems, and how the pattern solves it.

Post/Redirect/Get patterns are crucial to maintaining orders for form submissions on the Server side. Without it, the Server wouldn't be able to tell whether a user with certain Authorization should be accessing certain parts of the Web Application. Some examples include a logged in user to access the registration form for the Web Server and vice versa with a non-registered user accessing a registered user's dashboards and such. Another specific example is after the purchase of an item from a store. If a user refreshes the page, and the method of request is still a POST instead of a GET request, it might have unintended consequences such as sending another purchase request of the item again. Making sure it's a GET request redirection instead allows the user to freely refresh the pages without worrying about buying another item.

5. (10 marks) Consider the following application architecture. The web server queries the database and server-side renders HTML pages. Session-based authentication is used. At this time, session data is held by the web server.

Due to the increased popularity of the application, the web server is now running at over 70% CPU utilization during peak hours, and the database is consistently running at over 50% CPU utilization with spikes up to 80% at times. Approximately 75% of database queries are reads (select statements).

Discuss the limitations that this architecture has, and provide specific recommendations for maximizing the scalability of the system.



The limitations that are in place for such a scenario are the available processing power of the Web Server and the Database. Without secondary memory storage, such as Redis, any DB queries might pile up and slow down the results as many users ask for Read request that could scan the entirety of the database. Another is the use of multiple Web Servers to host requests as making a fail safe for the event that 1 server goes down; the whole process doesn't completely shut down. In addition, this would in turn lower the strain on HTTP Requests for a single Web Server. For a slightly expensive overhaul, another change that could be put in place is to just increase the physical limitations of the systems (RAM, CPU upgrades, etc.)

6. (10 marks) Describe the issues with the following code, and how to fix them. Note that the name of the file below is **register.php**. There are multiple security vulnerabilities and various bad practices.

This question assumes that a database exists named “db_name_here” with no password, and that there is a “users” table with “username” and “password” columns on the table.

register.php:

```
1 <?php
2 if ($_SERVER['REQUEST_METHOD'] === 'GET') {
3     if (isset($_GET['password']) && isset($_GET['username'])) {
4         $password = md5($_GET['password']);
5         $username = $_GET['username'];
6         $mysqli = new mysqli("localhost", "root", "", "db_name_here", 3306);
7         $mysqli->query("INSERT INTO users (username, password) VALUES ('{$username}', '{$password}')");
8         header("Location: index.php?name=$username");
9     }
10 }
11 ?>
12 <!DOCTYPE html>
13 <html lang="en">
14 <head>
15     <title>C3015 Question</title>
16 </head>
17 <body>
18     <form action="register.php" method="GET">
19         <label for="username">Username:</label>
20         <input id="username" type="text" name="username">
21         <label for="password">Password:</label>
22         <input id="password" type="text" name="password">
23         <input type="submit" value="Register">
24     </form>
25 </body>
26 </html>
27
```

Describe the issues below, and how to fix them in as much detail as you can.

1. Type="text" for passwords should just be type="password" unless you want the input password to be visible.
2. Registrations should be a POST request, not a GET.
3. Using md5 encryption isn't recommended for passwords as it's fragile and easy to break. Use Bcrypt or AES, just something more secure.
4. There's no validation of the client's input other than being set. (XXS and SQL injections). Add some checks with if/else statements or try catches.